

2.3

设文法 G 为

$$\begin{aligned} R &\rightarrow XRX \mid S, \\ S &\rightarrow aTb \mid bTa, \\ T &\rightarrow XTX \mid X \mid \varepsilon, \\ X &\rightarrow a \mid b. \end{aligned}$$

a. G 的变元为

$$\{R, S, T, X\}.$$

b. G 的终结符为

$$\{a, b\}.$$

c. G 的起始变元为

$$R.$$

d. $L(G)$ 中的三个字符串可以取为

$$ab, \quad ba, \quad aab.$$

e. 不在 $L(G)$ 中的三个字符串可以取为

$$\varepsilon, \quad a, \quad b.$$

f. 判断 $T \Rightarrow aba$ 是否为真。

不真。因为 T 的一步推导只能是

$$T \Rightarrow XTX, \quad T \Rightarrow X, \quad T \Rightarrow \varepsilon,$$

不能一步推出 aba 。

g. 判断 $T \Rightarrow^* aba$ 是否为真。

为真。例如：

$$T \Rightarrow XTX \Rightarrow aTb \Rightarrow aXb \Rightarrow abb$$

可以推出长度为 3 的串。对于 aba ，有

$$T \Rightarrow XTX \Rightarrow aTa \Rightarrow aXa \Rightarrow aba.$$

h. 判断 $T \Rightarrow T$ 是否为真。

不真。文法中没有产生式 $T \rightarrow T$ 。

i. 判断 $T \Rightarrow^* T$ 是否为真。

为真。因为 $\Rightarrow^*$ 允许零步推导，所以

$$T \Rightarrow^* T.$$

j. 判断 $XXX \Rightarrow^* aba$ 是否为真。

为真。因为

$$XXX \Rightarrow aXX \Rightarrow abX \Rightarrow aba.$$

k. 判断 $X \Rightarrow^* aba$ 是否为真。

不真。因为 X 只能推出单个终结符：

$$X \Rightarrow a \quad \text{或} \quad X \Rightarrow b.$$

l. 判断 $T \Rightarrow^* XX$ 是否为真。

为真。例如：

$$T \Rightarrow XTX \Rightarrow X\epsilon X = XX.$$

m. 判断 $T \Rightarrow^* XXX$ 是否为真。

为真。例如：

$$T \Rightarrow XTX \Rightarrow XXX.$$

n. 判断 $S \Rightarrow^* \epsilon$ 是否为真。

不真。因为

$$S \rightarrow aTb \quad \text{或} \quad S \rightarrow bTa,$$

无论 T 如何推导，最终串至少含有两个终结符。

o. 用普通语言描述 $L(G)$ 。

T 可以生成任意由 a, b 构成的字符串； S 生成首尾符号不同的字符串； R 可以在这样的字符串两端继续添加任意符号。因此

$$L(G)$$

正是所有非回文的 a, b 字符串。

2.6

给出产生下述语言的上下文无关文法。

a. 字母表 $\{a, b\}$ 上 a 多于 b 的所有字符串组成的集合。

构造文法

$$S \rightarrow TaT,$$

$$T \rightarrow TT \mid aTb \mid bTa \mid a \mid \epsilon.$$

其中， T 生成所有满足 a 的个数不少于 b 的个数的字符串。于是 $S \rightarrow TaT$ 额外增加一个 a ，从而保证整个字符串中 a 的个数严格多于 b 的个数。

b. 语言

$$\{a^n b^n \mid n \geq 0\}$$

的补集。

其补集由两类字符串组成：

第一类是不属于 a^*b^* 的字符串，即含有子串 ba 的字符串；

第二类是形如 $a^i b^j$ ，但 $i \neq j$ 的字符串。

构造文法

$$S \rightarrow UbaU \mid A \mid B,$$

$$U \rightarrow aU \mid bU \mid \varepsilon,$$

$$A \rightarrow aAb \mid aA \mid a,$$

$$B \rightarrow aBb \mid Bb \mid b.$$

其中， $UbaU$ 生成所有含有子串 ba 的字符串； A 生成形如 $a^i b^j$ 且 $i > j$ 的字符串； B 生成形如 $a^i b^j$ 且 $i < j$ 的字符串。

c.

$$\{w\#x \mid w, x \in \{0, 1\}^*, \text{ 且 } w^R \text{ 是 } x \text{ 的子串}\}.$$

构造文法

$$S \rightarrow TX,$$

$$T \rightarrow 0T0 \mid 1T1 \mid \#X,$$

$$X \rightarrow 0X \mid 1X \mid \varepsilon.$$

其中， T 负责生成

$$w\#yw^R$$

的形式，最后的 X 再生成任意后缀，因此整体可以生成

$$w\#yw^R z.$$

这正表示 w^R 是 $x = yw^R z$ 的子串。

d.

$$\{x_1\#x_2\#\cdots\#x_k \mid k \geq 1, x_i \in \{a, b\}^*, \text{ 且存在 } i, j \text{ 使得 } x_i = x_j^R\}.$$

下面按照题干未要求 $i \neq j$ 的通常理解给出文法。

设

$$W \rightarrow aW \mid bW \mid \varepsilon,$$

其中 W 生成任意一个块。

令

$$P \rightarrow \varepsilon \mid W\#P,$$

$$Q \rightarrow \varepsilon \mid \#WQ.$$

其中 P 生成被选中块之前的若干块， Q 生成被选中块之后的若干块。

首先生成两个互为反转的块：

$$S_{\text{pair}} \rightarrow PCQ,$$

$$C \rightarrow aCa \mid bCb \mid M,$$

$$M \rightarrow \# \mid \#WM.$$

其中, C 生成

$$u\#(\text{若干中间块})\#u^R$$

或相邻的

$$u\#u^R.$$

若允许 $i = j$, 还需要允许某一个块本身是回文串:

$$S_{\text{pal}} \rightarrow PHQ,$$

$$H \rightarrow aHa \mid bHb \mid a \mid b \mid \varepsilon.$$

最终文法为

$$S \rightarrow S_{\text{pair}} \mid S_{\text{pal}}.$$

2.7

给出关于练习 2.6 中语言的 PDA 的非形式描述。

- a. 字母表 $\{a, b\}$ 上 a 多于 b 的所有字符串组成的集合。

PDA 用栈记录当前未被抵消的字符。

扫描输入时:

- 如果读到 a , 而栈顶是 b , 则弹出这个 b ;
- 如果读到 b , 而栈顶是 a , 则弹出这个 a ;
- 如果读到的字符与栈顶相同, 或者栈为空, 则把该字符压入栈。

扫描结束后, 如果栈中剩余的是至少一个 a , 则说明 a 的个数多于 b 的个数, PDA 接受; 否则拒绝。

- b. 语言

$$\{a^n b^n \mid n \geq 0\}$$

的补集。

PDA 分两种情况处理。

第一种情况: 如果输入中出现了子串 ba , 则该串不可能属于 a^*b^* , 因此直接接受。

第二种情况: 输入形如 $a^i b^j$ 。PDA 在读到 a 时把一个栈符号压入栈; 进入 b 阶段后, 每读到一个 b 就弹出一个栈符号。

- 如果读 b 时栈已经空, 说明 $j > i$, 接受;
- 如果输入结束时栈中仍有符号, 说明 $i > j$, 接受;
- 只有当输入结束且栈正好为空时, 说明 $i = j$, 拒绝。

因此该 PDA 正好识别

$$\{a^n b^n \mid n \geq 0\}$$

的补集。

c.

$$\{w\#x \mid w, x \in \{0, 1\}^*, \text{ 且 } w^R \text{ 是 } x \text{ 的子串}\}.$$

PDA 在读到符号 # 之前，将 w 的字符依次压入栈。这样，栈顶到栈底的顺序正好对应 w^R 。

读到 # 后，PDA 开始扫描 x 。它可以非确定性地选择 x 中的某一个位置作为匹配 w^R 的开始位置。

从该位置起，PDA 将输入字符与栈顶字符逐一比较：

- 如果相同，则弹出栈顶并继续；
- 如果不同，则该分支拒绝；
- 如果栈被清空，说明已经在 x 中找到了子串 w^R ，于是 PDA 读完剩余输入并接受。

如果不存在符号 #，或者所有非确定性分支都无法清空栈，则拒绝。

d.

$$\{x_1\#x_2\#\cdots\#x_k \mid k \geq 1, x_i \in \{a, b\}^*, \text{ 且存在 } i, j \text{ 使得 } x_i = x_j^R\}.$$

PDA 非确定性地选择某一个块 x_i ，并在读这个块时把它的字符依次压入栈。读完该块之后，PDA 继续向后扫描输入，并非确定性地选择某一个后面的块 x_j 与栈中内容进行比较。

比较时，PDA 每读入 x_j 的一个字符，就检查它是否等于栈顶字符：

- 若相等，则弹出栈顶；
- 若不相等，则该分支拒绝；
- 若在 x_j 正好结束时栈被清空，则说明 $x_j = x_i^R$ ，接受。

如果题目允许 $i = j$ ，则 PDA 还可以增加一个分支：非确定性地选择某个块，并用标准的回文语言 PDA 方法检查该块是否为回文串。若是回文串，则该块满足

$$x_i = x_i^R,$$

因此接受。

2.10

给出识别练习 2.9 中语言

$$A = \{a^i b^j c^k \mid i, j, k \geq 0, \text{ 且 } i = j \text{ 或 } j = k\}$$

的 PDA 的非形式化描述。

该语言可以看作两个语言的并：

$$A_1 = \{a^i b^j c^k \mid i, j, k \geq 0, i = j\},$$

$$A_2 = \{a^i b^j c^k \mid i, j, k \geq 0, j = k\}.$$

PDA 在开始时非确定性地选择检查 $i = j$ 或检查 $j = k$ 。

(1) 检查 $i = j$ 的分支。

PDA 首先读取所有 a ，每读到一个 a 就向栈中压入一个栈符号。随后进入 b 阶段，每读到一个 b 就弹出一个栈符号。

如果在读 b 时栈中已经没有可弹出的 a 标记，则说明 $j > i$ ，该分支拒绝；如果 b 阶段结束后栈中仍有未弹出的 a 标记，则说明 $i > j$ ，该分支拒绝。只有当 b 阶段结束时栈恰好回到底标记，才说明 $i = j$ 。之后 PDA 读完任意多个 c 并接受。

(2) 检查 $j = k$ 的分支。

PDA 首先读取并忽略所有 a 。随后进入 b 阶段，每读到一个 b 就向栈中压入一个栈符号。进入 c 阶段后，每读到一个 c 就弹出一个栈符号。

如果在读 c 时栈中已经没有可弹出的 b 标记，则说明 $k > j$ ，该分支拒绝；如果输入结束后栈中仍有未弹出的 b 标记，则说明 $j > k$ ，该分支拒绝。只有当输入结束时栈恰好回到底标记，才说明 $j = k$ ，于是接受。

两个分支都要求输入必须保持 $a^*b^*c^*$ 的形式。如果出现顺序错误，例如在 b 之后又出现 a ，或者在 c 之后又出现 a 或 b ，则相应分支拒绝。

2.23

证明 DCFL 类在下列运算下不是封闭的。

记

$$H_1 = \{a^n b^n c^m \mid n, m \geq 1\},$$

$$H_2 = \{a^m b^n c^n \mid m, n \geq 1\},$$

并令

$$H = H_1 \cup H_2.$$

其中 H_1 和 H_2 都是 DCFL，但

$$H = \{a^i b^j c^k \mid i, j, k \geq 1, i = j \text{ 或 } j = k\}$$

不是 DCFL。这是练习 2.9 中经典非确定上下文无关语言的正长度版本；它是上下文无关语言，但不是确定上下文无关语言。

a. 并。

若 DCFL 对并封闭，由于 DCFL 对补封闭，则 DCFL 也会对交封闭。事实上，

$$A \cap B = \overline{\overline{A} \cup \overline{B}}.$$

但下面将说明 DCFL 对交不封闭，因此 DCFL 对并也不封闭。

b. 交。

考虑

$$L_1 \cap L_2.$$

有

$$L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\}.$$

而

$$\{a^n b^n c^n \mid n \geq 0\}$$

不是上下文无关语言，因此更不可能是 DCFL。

所以 DCFL 对交不封闭。

c. 连接。

取

$$A = \{\varepsilon, d\},$$

$$B = H_1 \cup dH_2.$$

其中 A 是正则语言，因此是 DCFL。语言 B 也是 DCFL，因为确定 PDA 可以根据输入的第一个符号判断应当进入哪个分支：若输入以 d 开头，则去掉 d 后检查 H_2 ；若输入以 a 开头，则检查 H_1 。

假设 DCFL 对连接封闭，则

$$AB$$

也应当是 DCFL。注意

$$AB = H_1 \cup dH_2 \cup dH_1 \cup ddH_2.$$

再将其与正则语言

$$da^+b^+c^+$$

取交。由于 DCFL 对正则语言取交封闭，若 AB 是 DCFL，则

$$AB \cap da^+b^+c^+$$

也应当是 DCFL。

但是

$$AB \cap da^+b^+c^+ = dH_1 \cup dH_2 = dH.$$

如果 dH 是 DCFL，那么去掉固定前缀 d 后， H 也应当是 DCFL。这与 H 不是 DCFL 矛盾。

因此，DCFL 对连接运算不封闭。

d. 星号。

取

$$C = \{d\} \cup H_1 \cup dH_2.$$

语言 C 是 DCFL。确定 PDA 可以根据输入开头进行判断：若输入是单独的 d ，则接受；若输入以 a 开头，则检查 H_1 ；若输入以 d 后接 a 开头，则去掉前导 d 后检查 H_2 。

假设 DCFL 对星号封闭，则

$$C^*$$

也应当是 DCFL。考虑它与正则语言

$$da^+b^+c^+$$

的交。由于 DCFL 对正则语言取交封闭，若 C^* 是 DCFL，则

$$C^* \cap da^+b^+c^+$$

也应当是 DCFL。

下面计算该交集。要得到形如

$$da^+b^+c^+$$

的字符串，在 C^* 中只有两种可能：

- 先取因子 d ，再取一个 H_1 中的字符串，得到 dH_1 ；
- 直接取一个 dH_2 中的字符串，得到 dH_2 。

因此

$$C^* \cap da^+b^+c^+ = dH_1 \cup dH_2 = dH.$$

若该语言是 DCFL，则去掉固定前缀 d 后， H 也应当是 DCFL，矛盾。

因此，DCFL 对星号运算不封闭。

e. 逆。

取新的标记符号 d, e ，并令

$$K = dH_1^R \cup eH_2^R.$$

语言 K 是 DCFL。因为确定 PDA 读入第一个符号后即可确定应该检查哪一种条件：

- 若开头是 d ，则接下来检查 H_1^R ，也就是检查形如 $c^mb^na^n$ 的字符串中 b 与 a 的个数是否相等；
- 若开头是 e ，则接下来检查 H_2^R ，也就是检查形如 $c^nb^na^m$ 的字符串中 c 与 b 的个数是否相等。

假设 DCFL 对逆运算封闭，则

$$K^R$$

也应当是 DCFL。可是

$$K^R = H_1d \cup H_2e.$$

直观上，确定 PDA 在读入前面的 $a^+b^+c^+$ 部分时，还没有看到最后的标记 d 或 e ，因此无法确定应该比较 a 与 b 的个数，还是比较 b 与 c 的个数。这正是确定性失效的地方。

更形式化地说，若

$$H_1d \cup H_2e$$

是 DCFL，则对右端的有限标记集合 $\{d, e\}$ 作右商后，也应得到 DCFL：

$$(H_1d \cup H_2e)/\{d, e\} = H_1 \cup H_2 = H.$$

但 H 不是 DCFL，矛盾。

因此 K^R 不是 DCFL，从而 DCFL 对逆运算不封闭。

2.35

设

$$D = \{xy \mid x, y \in \{0, 1\}^*, |x| = |y| \text{ 但 } x \neq y\}.$$

证明 D 是上下文无关语言。

我们构造一个上下文无关文法生成 D 。

令 C 生成任意一个二进制符号：

$$C \rightarrow 0 \mid 1.$$

令 A_0 生成所有奇数长度且正中间符号为 0 的字符串：

$$A_0 \rightarrow 0 \mid CA_0C.$$

令 A_1 生成所有奇数长度且正中间符号为 1 的字符串：

$$A_1 \rightarrow 1 \mid CA_1C.$$

定义起始变元 S ：

$$S \rightarrow A_0A_1 \mid A_1A_0.$$

下面说明该文法生成的正是 D 。

若

$$S \Rightarrow A_0A_1,$$

则前一部分是奇数长度且中间符号为 0 的字符串，后一部分是奇数长度且中间符号为 1 的字符串。设前一部分长度为 $2i + 1$ ，后一部分长度为 $2j + 1$ ，则整个字符串长度为

$$2(i + j + 1),$$

其前半部分和后半部分长度相等。并且， A_0 的中心符号和 A_1 的中心符号在整个字符串中相距正好一半长度，因此它们分别位于前半串和后半串的对应位置上。由于一个为 0，一个为 1，所以两半不同。

同理，

$$S \Rightarrow A_1A_0$$

时，两半也在某个对应位置上不同。

反过来，若 $xy \in D$ ，则 $|x| = |y|$ 且 $x \neq y$ 。因此存在某个位置，使得 x 和 y 在该对应位置上的符号不同。若该位置上 x 中是 0， y 中是 1，则该字符串可由 A_0A_1 生成；若该位置上 x 中是 1， y 中是 0，则可由 A_1A_0 生成。

因此 D 是上下文无关语言。